



HTTP Requests

Persistent Interactive | Persistent University



Key Learning Points

- How to make HTTP Requests from Angular
- How to Import the HttpClientModule
- Use of Observables
- Passing header parameters

Import the HttpClientModule

- In the module.ts file, import the HttpClientModule

```
import { HttpClientModule } from '@angular/common/http';
```

- Add to imports Array

```
@NgModule({  
  declarations: [],  
  imports: [  
    BrowserModule,  
    AppRoutingModule,  
    HttpClientModule,  
  ],  
  providers: []  
})  
export class AppModule {  
}
```

Methods of Http Service

1. **get**(url: string, options?: RequestOptionsArgs): **Observable**<Response>;
 2. **post**(url: string, body:any, options?: RequestOptionsArgs): **Observable**<Response>;
 3. **put**(url: string, body:any,options?: RequestOptionsArgs): **Observable**<Response>;
 4. **delete**(url: string, options?: RequestOptionsArgs): **Observable**<Response>;
- Performs http requests using **XMLHttpRequest** as the default backend.
 - All the methods return an Observable type.

Inject the Http service

- As a best practice, make HTTP calls from a service and not in a component.
- Create a Service named “**UserService**”
- Inject the **HttpClient** service in the constructor.

```
export class UserService {  
  constructor(private http: HttpClient) {  
  
  }  
}
```

- Add the import statement accordingly.

```
import { HttpClient } from "@angular/common/http";
```

Define an Observable

- **Define `getUserList()` to fetch list of users.**

```
getUserList(): Observable<any> {  
  return this.http.get('https://jsonplaceholder.typicode.com/users');  
}
```

- **Define another method to fetch single user by passing `userid`.**

```
getUserDetails(userid:number): Observable<any> {  
  return this.http.get('https://jsonplaceholder.typicode.com/users/'+userid);  
}
```

Typing the returned data

- Define `getUserList()` to fetch list of typed users.

```
getUserList(): Observable<User[]> {  
    return this.http.get<User[]>('https://jsonplaceholder.typicode.com/users');  
}
```

- Define another method to fetch single typed user by passing `userid`.

```
getUserDetails(userid:number): Observable<User> {  
    return this.http.get< User>('https://jsonplaceholder.typicode.com/users/'+userid);  
}
```

Inject the UserService

- Inject the UserService in the UserListComponent.
- Define an Array of users in the component which will be fetched from the service.

```
export class UserlistComponent implements OnInit {  
  userList: Array<any>;  
  
  constructor(private userservice:UserService){  
  }  
  
}
```


Subscribe to the Observable

- Subscribe to the Observable in the UserListComponent

```
export class UserlistComponent implements OnInit {  
  userList: Array<any>;  
  
  ngOnInit() {  
    this.userservice.getUserList().subscribe(data=>{  
      this.userList = data;  
    })  
  }  
}
```

Converting Observables to Promise

- **Convert Observable to Promise with below method.**

```
getUserListAsPromise(): Promise<any> {  
    return this.http.get('https://jsonplaceholder.typicode.com/users').toPromise();  
}
```

- **Consuming the Promise in the component.**

```
this.userservice.getUserListAsPromise().then(data=>{  
    this.userList = data;  
});
```

Async pipe

- The code of subscription can be further optimized by using the async pipe.
- Few changes to the code as given below:

1. Change the type of data to Observable

userList: Observable<any>;

2. Change the code of subscription. No need to write subscribe code.

this.userList = this.userservice.getUserList()

3. Use Async pipe in the template

<li *ngFor="let user of userList |async">

UnSubscribe to a subscription

- UnSubscribe and Cancel a request.
- Subscription.dispose().

```
if ( this.subscription ) {  
  this.subscription.dispose();  
}  
this.subscription = this.http.get( 'awesomeApi' )  
  .subscribe((res)=> {  
    // your awesome code..  
  })
```

Setting request headers

- Import statements

```
import { Headers, RequestOptions } from '@angular/common/http';
```

- Set values for the Header object

```
let headers = new Headers({ 'Content-Type':  
'application/json' });  
let options = new RequestOptions({ headers: headers });
```

- Pass the options object in the Http call.

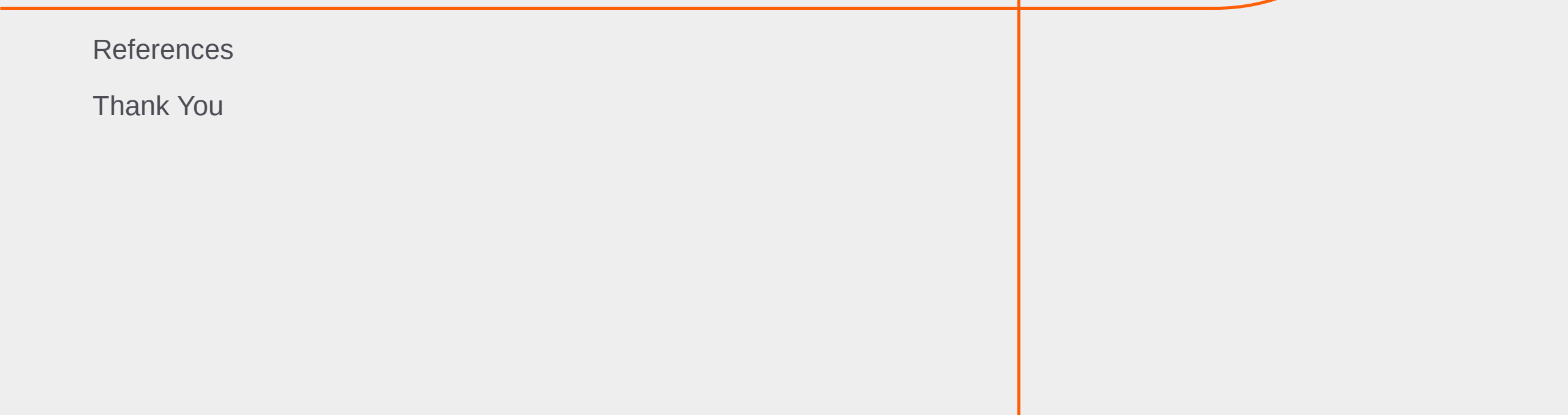
```
this.http.post(this.Url, { name }, options)
```

Summary: Session

With this we have come to an end of our session, where we discussed about

- How to make HTTP calls from Angular
- Using the HttpClient service
- Making a GET request
- Making a POST request

Appendix



References

Thank You

Key Contacts :

Persistent University :

- Archana Ghatigar
archana_ghatigar@persistent.co.in



Thank you!

Persistent Interactive | Persistent University

